Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Due Date	27-01-2026		

Experiment 3: Regression Analysis using Linear and Regularized Models

1 Objective

To implement linear and regularized regression models for predicting a continuous target variable, evaluate their performance using multiple metrics, visualize model behavior, and analyze overfitting, underfitting, and bias–variance characteristics.

2 Dataset Description

A real-world regression dataset containing numerical and categorical features related to loan applications is used. The target variable is the **loan amount sanctioned**.

3 Preprocessing Steps

Based on the implementation, the following preprocessing steps were carried out:

1. **Data Cleaning**: Duplicate rows were identified using `df.duplicated()` and removed with `df.drop_duplicates()`.
2. **Missing Values**: Handled by checking for null entries using `df.isnull().sum()` and removed missing values using SimpleImputer where used median for numerical features and mode for categorical features
3. **Feature Selection**: Dropped the unnecessary features from the dataframe using `df.drop(columns = col, inplace = 'True')`
4. **Handling outliers**: Handled the outliers using capping. `df.isnull().sum()`
5. **Encoding Categorical Variables**: Encoded categorical features using LabelEncoder.
6. **Feature Scaling**:
7. **Feature Scaling**: Since Linear Regression and its regularized variants (Ridge, Lasso, and Elastic Net) are sensitive to feature magnitudes, `StandardScaler` was applied to normalize the input features. This ensures that all variables contribute equally to the model, prevents dominance of large-scale features, and enables effective regularization during model training.
8. **Data Splitting**: The dataset was split into training and testing sets using `train_test_split`.

4 Implementation Details

The following code snippets illustrate the core implementation of the classifiers:

Important Libraries to be included

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso
from sklearn.linear_model import ElasticNet
from sklearn.preprocessing import RobustScaler
import time
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import math
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from matplotlib import cm
from mpl_toolkits.mplot3d import Axes3D

plt.rcParams.update({
    "font.family": "Arial",
    "font.size": 15,
    "axes.titlesize": 15,
    "axes.titleweight": "bold",
    "axes.labelsize": 15,
    "axes.labelweight": "bold",
    "xtick.labelsize": 13,
    "ytick.labelsize": 13
})
```

1. Load the dataset

```
# loading dataset
df_train = pd.read_csv("train.csv")
df_test = pd.read_csv("test.csv")

# Seeing first 5 rows.
df_train.head()
df_test.head()

# Checking the data types of each features of the dataset
df_train.info()
```

2. Perform data preprocessing (handling missing values and scaling)

```
# Checking if any duplicate rows are found.
print("Duplicates in train data: ", any(df_train.duplicated()))
print("Duplicates in test data: ", any(df_test.duplicated()))
# ---- > No duplicates

## Remove unnecessary columns such as Customer ID, Name
cols_to_drop = ['Customer ID', 'Name']
df_train.drop(columns=cols_to_drop, inplace=True)
df_test.drop(columns=cols_to_drop, inplace=True)

# Remove rows with negative Loan Amount
df_train = df_train[df_train["Loan Sanction Amount (USD)"] > 0]

# Checking the missing and null values...for train data
df_train.isnull().sum()

# Checking the missing and null values...for test data
df_test.isnull().sum()

# Handle Missing Values
# dropping rows which has target feature missing
df_train = df_train.dropna(subset=['Loan Sanction Amount (USD)'])

# Numerical Features     Mean
num_train_cols = df_train.select_dtypes(include=["int64","float64"]).columns
num_imputer = SimpleImputer(strategy="median")
df_train[num_train_cols] = num_imputer.fit_transform(df_train[num_train_cols])
num_test_cols = df_test.select_dtypes(include=["int64","float64"]).columns
num_imputer = SimpleImputer(strategy="median")
df_test[num_test_cols] = num_imputer.fit_transform(df_test[num_test_cols])

# Categorical Features     Mode
cat_cols = df_train.select_dtypes(include=["object"]).columns
cat_imputer = SimpleImputer(strategy="most_frequent")
df_train[cat_cols] = cat_imputer.fit_transform(df_train[cat_cols])
df_test[cat_cols] = cat_imputer.transform(df_test[cat_cols])

# Encoding Categorical Variables
le = LabelEncoder()
for col in df_train.select_dtypes(include='object'):
    all_values = pd.concat([df_train[col], df_test[col]])
    le.fit(all_values)
    df_train[col] = le.transform(df_train[col])
    df_test[col] = le.transform(df_test[col])

# Separate features (X) and target (y)
X = df_train.drop("Loan Sanction Amount (USD)", axis=1)
y = df_train["Loan Sanction Amount (USD)"]
```

3. Perform Exploratory Data Analysis (EDA)

```
# Statistical summary for train data and test data
df_train.describe()
df_test.describe()

print("Shape of Train dataset: ",df_train.shape)
print("Shape of Test dataset: ",df_test.shape)
```

```
# Correlation analysis
corr = df_train.corr()
corr

# Boxplot for outliers.
cols_per_plot = 10  # number of features in each subplot
total_cols = X.shape[1]
num_plots = math.ceil(total_cols / cols_per_plot)
fig, axes = plt.subplots(nrows=num_plots, ncols=1, figsize=(16, num_plots*4))
for i in range(num_plots):
    start_col = i * cols_per_plot
    end_col = min((i+1) * cols_per_plot, total_cols)
    subset = X.iloc[:, start_col:end_col]  # select subset of columns

    sns.boxplot(data=subset, ax=axes[i])
    axes[i].set_title(f"Boxplot for Features {start_col+1} to {end_col}")
    axes[i].set_xlabel("Features")
    axes[i].set_ylabel("Values")
    axes[i].tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()

# Handling outliers by using capping.
upper_limit = df_train['Loan Sanction Amount (USD)'].quantile(0.95)
lower_limit = df_train['Loan Sanction Amount (USD)'].quantile(0.05)
df_train['Loan Sanction Amount (USD)'] = np.where(df_train['Loan Sanction Amount (
    USD)']>upper_limit, upper_limit, np.where(df_train['Loan Sanction Amount (USD)'
    ]<lower_limit, lower_limit, df_train['Loan Sanction Amount (USD)']))

# Heatmap plot for correlation analysis
plt.figure(figsize=(12, 10))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    square=True,
    annot_kws={"size": 8}
)
plt.title("Correlation Heatmap")
plt.show()
```

4. Visualize feature distributions and target distribution

```
# Target variable distribution plot
plt.figure()
sns.histplot(y, bins=20, kde=True)
plt.title("Target Variable Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.show()

# Feature vs. target scatter plots
# Select numerical features
num_features = df_train.select_dtypes(include=['int64', 'float64']).columns
# Number of columns
```

```
cols = 3
rows = math.ceil(len(num_features) / cols)
# Create subplots
fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 4*rows))
axes = axes.flatten()
# Plot each feature
for i, col in enumerate(num_features):
    sns.scatterplot(x=df_train[col], y=df_train["Loan Sanction Amount (USD)"], ax=
    axes[i])
    axes[i].set_title(f"{col} vs Target")
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Loan Amount")
# Remove empty subplots
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()
```

5. Split the dataset into training and testing sets

```
# Split the dataset into training and testing sets
X_train, X_val, y_train, y_val = train_test_split( X, y, test_size=0.2,
    random_state=42)
```

6. Train baseline Linear Regression

```
# Standardize numerical features
scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(df_test)

# Base linear regression
reg = LinearRegression()
start_time = time.time()
reg.fit(X_train_scaled, y_train)
end_time = time.time()
linear_train_time = end_time - start_time

y_val_pred = reg.predict(X_val_scaled)
mae = mean_absolute_error(y_val, y_val_pred)
mse = mean_squared_error(y_val, y_val_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_val, y_val_pred)

# Prediction for the test data and storing the predictions as csv file.
test_predictions = reg.predict(X_test_scaled)
submission = pd.DataFrame({
    "Loan Sanction Amount (USD)": test_predictions
})
submission.to_csv("submission.csv", index=False)
```

7. Train Ridge, Lasso, and Elastic Net models

```python
# Ridge Regression
ridge_model = Ridge(alpha=1.0)  # default regularization
start_time = time.time()
ridge_model.fit(X_train_scaled, y_train)
end_time = time.time()
ridge_train_time = end_time - start_time
y_val_pred_ridge = ridge_model.predict(X_val_scaled)
ridge_mae = mean_absolute_error(y_val, y_val_pred_ridge)
ridge_mse = mean_squared_error(y_val, y_val_pred_ridge)
ridge_rmse = np.sqrt(ridge_mse)
ridge_r2 = r2_score(y_val, y_val_pred_ridge)

# Lasso Regression
lasso_model = Lasso(alpha=0.01, max_iter=10000)
start_time = time.time()
lasso_model.fit(X_train_scaled, y_train)
end_time = time.time()
lasso_train_time = end_time - start_time
y_val_pred_lasso = lasso_model.predict(X_val_scaled)
lasso_mae = mean_absolute_error(y_val, y_val_pred_lasso)
lasso_mse = mean_squared_error(y_val, y_val_pred_lasso)
lasso_rmse = np.sqrt(lasso_mse)
lasso_r2 = r2_score(y_val, y_val_pred_lasso)

# Elastic Net Regression
elastic_model = ElasticNet(alpha=0.1, l1_ratio=0.5, max_iter=10000)
start_time = time.time()
elastic_model.fit(X_train_scaled, y_train)
end_time = time.time()
elastic_train_time = end_time - start_time
y_val_pred_elastic = elastic_model.predict(X_val_scaled)
elastic_mae = mean_absolute_error(y_val, y_val_pred_elastic)
elastic_mse = mean_squared_error(y_val, y_val_pred_elastic)
elastic_rmse = np.sqrt(elastic_mse)
elastic_r2 = r2_score(y_val, y_val_pred_elastic)
```

8. Perform hyperparameter tuning using 5-Fold Cross-Validation

```python
# Hyperparameter Tuning for Ridge Regression - Grid Search CV
ridge = Ridge()
ridge_params = { "alpha": [0.01, 0.1, 1, 10, 100] }
ridge_grid = GridSearchCV( ridge, ridge_params, cv=5, scoring="r2")
ridge_grid.fit(X_train_scaled, y_train)

# Hyperparameter Tuning for Lasso Regression - Grid Search CV
lasso = Lasso(max_iter=50000)
lasso_params = {"alpha": [0.001, 0.01, 0.1, 1, 10] }
lasso_grid = GridSearchCV(lasso, lasso_params, cv=5, scoring="r2")
lasso_grid.fit(X_train_scaled, y_train)

# Hyperparameter Tuning for Elastic Net Regression - Grid Search CV
elastic = ElasticNet(max_iter=50000)
elastic_params = {"alpha": [0.01, 0.1, 1, 10], "l1_ratio": [0.2, 0.5, 0.8] }
elastic_grid = GridSearchCV(elastic, elastic_params, cv=5,scoring="r2" )
elastic_grid.fit(X_train_scaled, y_train)
```

9. Evaluate all models using regression metrics

```
# Get Best Tuned Models
best_ridge = ridge_grid.best_estimator_
best_lasso = lasso_grid.best_estimator_
best_elastic = elastic_grid.best_estimator_

# Predictions on Validation Set
y_val_pred_lr = reg.predict(X_val_scaled)
y_val_pred_ridge = best_ridge.predict(X_val_scaled)
y_val_pred_lasso = best_lasso.predict(X_val_scaled)
y_val_pred_elastic = best_elastic.predict(X_val_scaled)

# Define Evaluation Function
def evaluate_model(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    return mae, mse, rmse, r2

lr_metrics = evaluate_model(y_val, y_val_pred_lr)
ridge_metrics = evaluate_model(y_val, y_val_pred_ridge)
lasso_metrics = evaluate_model(y_val, y_val_pred_lasso)
elastic_metrics = evaluate_model(y_val, y_val_pred_elastic)

performance_table = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression", "
    Elastic Net"],
    "MAE": [lr_metrics[0], ridge_metrics[0], lasso_metrics[0], elastic_metrics
    [0]],
    "MSE": [lr_metrics[1], ridge_metrics[1], lasso_metrics[1], elastic_metrics
    [1]],
    "RMSE": [lr_metrics[2], ridge_metrics[2], lasso_metrics[2], elastic_metrics
    [2]],
    "R2 Score": [lr_metrics[3], ridge_metrics[3], lasso_metrics[3],
    elastic_metrics[3]],
    "Training_Time: ":[linear_train_time, ridge_train_time, lasso_train_time,
    elastic_train_time]
})
performance_table
best_model = performance_table.sort_values("R2 Score", ascending=False).iloc[0]
best_model
```

10. Visualizations code

```
# Predicted vs Actual Values Plot
plt.figure(figsize=(12, 12))
models = [
    ("Linear Regression", y_val_pred_lr),
    ("Ridge Regression", y_val_pred_ridge),
    ("Lasso Regression", y_val_pred_lasso),
    ("Elastic Net", y_val_pred_elastic)
]
for i, (title, y_pred) in enumerate(models, 1):
    plt.subplot(2, 2, i)
    plt.scatter(y_val, y_pred, alpha=0.6)
```

```
    plt.plot([y_val.min(), y_val.max()],[y_val.min(), y_val.max()],linestyle="--")
    plt.xlabel("Actual Loan Amount")
    plt.ylabel("Predicted Loan Amount")
    plt.title(f"Predicted vs Actual ({title})")
plt.tight_layout()
plt.show()

# Residual Plots
plt.figure(figsize=(12, 10))
for i, (title, y_pred) in enumerate(models, 1):
    residuals = y_val - y_pred
    plt.subplot(2, 2, i)
    plt.scatter(y_pred, residuals, alpha=0.6)
    plt.axhline(0, linestyle="--")
    plt.xlabel("Predicted Values")
    plt.ylabel("Residuals")
    plt.title(f"Residual Plot ({title})")
plt.tight_layout()
plt.show()

# Training Error vs Validation Error plot
train_pred = best_elastic.predict(X_train_scaled)
train_rmse = np.sqrt(mean_squared_error(y_train, train_pred))
val_rmse = np.sqrt(mean_squared_error(y_val, y_val_pred_elastic))
plt.figure(figsize=(6,4))
plt.bar(["Training RMSE", "Validation RMSE"], [train_rmse, val_rmse])
plt.title("Training vs Validation Error")
plt.ylabel("RMSE")
plt.show()

# Coefficients Comparison Bar Plot
coef_df = pd.DataFrame({
    "Feature": X_train.columns,
    "Linear": reg.coef_,
    "Ridge": best_ridge.coef_,
    "Lasso": best_lasso.coef_,
    "Elastic Net": best_elastic.coef_
})
coef_df.set_index("Feature").plot(kind="bar", figsize=(14,10))
plt.title("Coefficient Comparison Across Models")
plt.ylabel("Coefficient Value")
plt.xticks(rotation=45)
plt.show()

# Lambda vs Slope (  ) for Ridge & Lasso
ridge_lambdas = np.logspace(-2, 2, 6)
lasso_lambdas = np.logspace(-3, 1, 6)
ridge_slopes = [], lasso_slopes = []
for lam in ridge_lambdas:
    model = Ridge(alpha=lam)
    model.fit(X_train_scaled, y_train)
    ridge_slopes.append(model.coef_)
for lam in lasso_lambdas:
    model = Lasso(alpha=lam, max_iter=10000)
    model.fit(X_train_scaled, y_train)
    lasso_slopes.append(model.coef_)
ridge_slopes = np.array(ridge_slopes)
lasso_slopes = np.array(lasso_slopes)
num_features = min(5, ridge_slopes.shape[1])
```

```
plt.figure(figsize=(14, 5))

# Ridge
plt.subplot(1, 2, 1)
for i in range(num_features):
    plt.plot(ridge_lambdas, ridge_slopes[:, i], marker='o', label=f'Slope   {i+1}'
    )
plt.xscale('log')
plt.xlabel('Lambda (  )')
plt.ylabel('Slope (  )')
plt.title('Ridge Regression:    vs Slope')
plt.legend()
plt.grid(True, alpha=0.3)

# Lasso
plt.subplot(1, 2, 2)
for i in range(num_features):
    plt.plot(lasso_lambdas, lasso_slopes[:, i], marker='o', label=f'Slope   {i+1}'
    )
plt.xscale('log')
plt.xlabel('Lambda (  )')
plt.ylabel('Slope (  )')
plt.title('Lasso Regression:    vs Slope')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Elastic Net: Lambda1, Lambda2 vs Slopes
alpha = best_elastic.alpha          # already obtained from CV
l1_ratios = [0.2, 0.5, 0.8]          # simple, clear values
lambda1_vals = [], lambda2_vals = []
slope1_vals = [], slope2_vals = []
for l1 in l1_ratios:
    lambda1 = alpha * l1
    lambda2 = alpha * (1 - l1)
    model = ElasticNet(alpha=alpha, l1_ratio=l1, max_iter=10000)
    model.fit(X_train_scaled, y_train)
    lambda1_vals.append(lambda1)
    lambda2_vals.append(lambda2)
    slope1_vals.append(model.coef_[0])   #  1
    slope2_vals.append(model.coef_[1])   #  2

# ---- Plot ----
plt.figure(figsize=(7, 5))
plt.plot(lambda1_vals, slope1_vals, marker='o', label='Slope  1 ')
plt.plot(lambda1_vals, slope2_vals, marker='o', label='Slope  2 ')
plt.xlabel('Lambda1 (L1 Regularization)')
plt.ylabel('Slope')
plt.title('Elastic Net:  1  (with corresponding  2 ) vs Slopes')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# X vs slope Plots
plt.figure(figsize=(14, 10))
models = [
    ("Linear Regression", reg),
    ("Ridge Regression", best_ridge),
```

```
    ("Lasso Regression", best_lasso),
    ("Elastic Net", best_elastic)
]
for idx, (title, model) in enumerate(models, 1):
    plt.subplot(2, 2, idx)
    for i in range(min(3, X_train_scaled.shape[1])):
        contribution = X_train_scaled[:, i] * model.coef_[i]
        plt.scatter(
            X_train_scaled[:, i],
            contribution,
            alpha=0.5,
            label=f'  {i+1}'
        )
    plt.xlabel('Feature Value (X)')
    plt.ylabel('Slope    X')
    plt.title(f'X vs Slope Contribution ({title})')
    plt.legend()
    plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

5 Visualizations

This section presents the visual analysis of the model's performance, including target distribution, feature behavior, Hyperparameter Analysis and coefficients comparison results across all models.

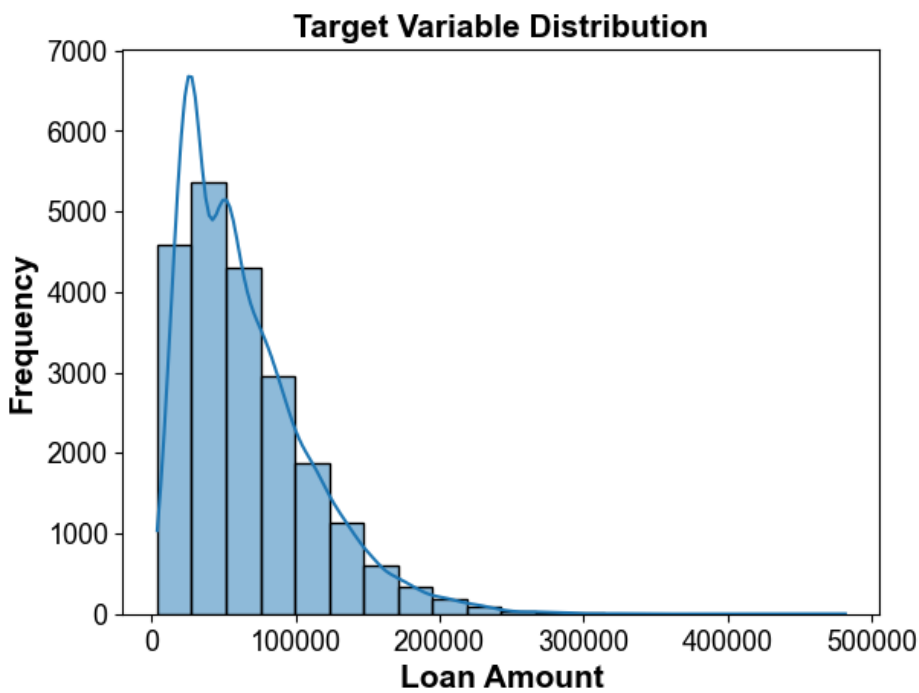

Figure 1: Target variable distribution plot

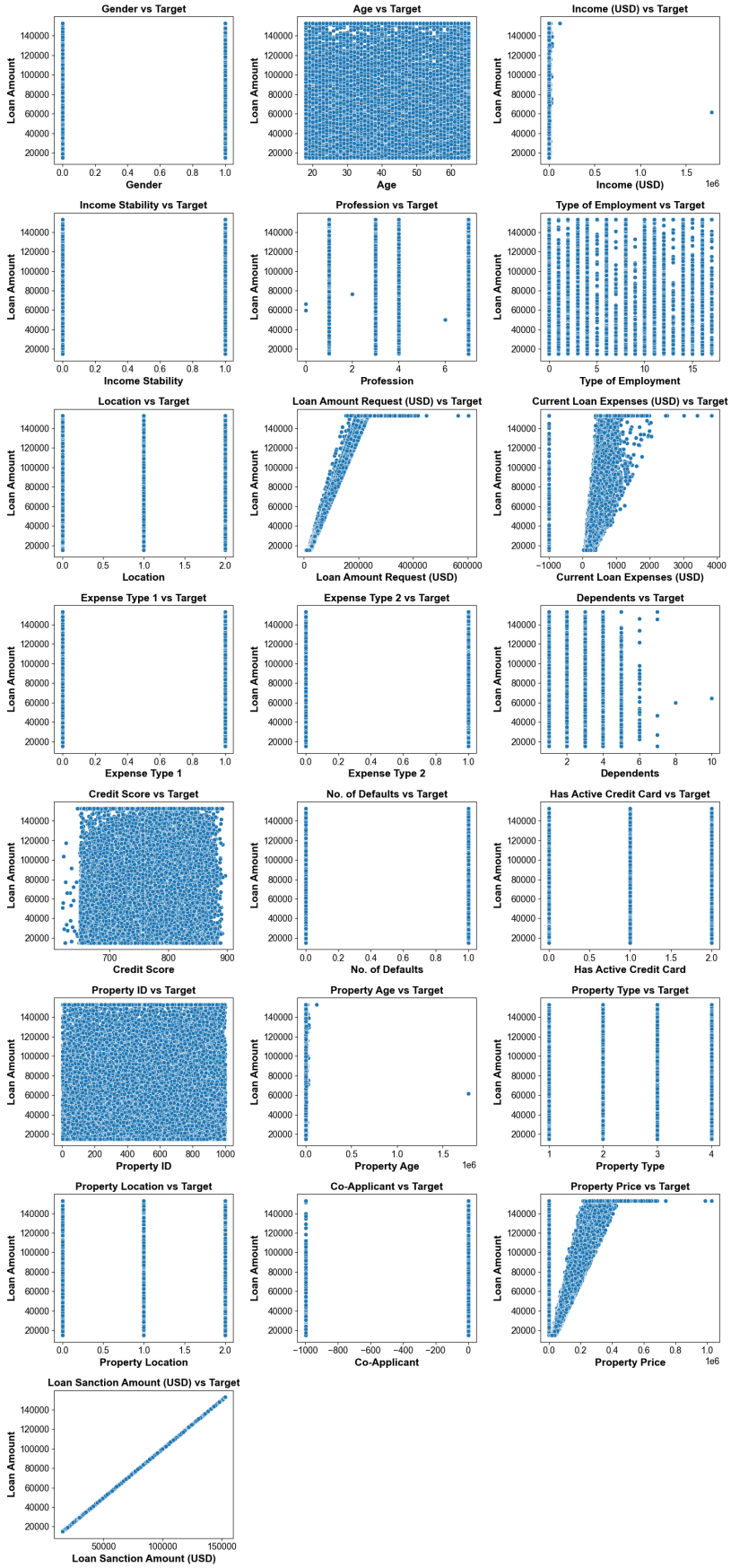

Figure 2: Feature vs. target scatter plots

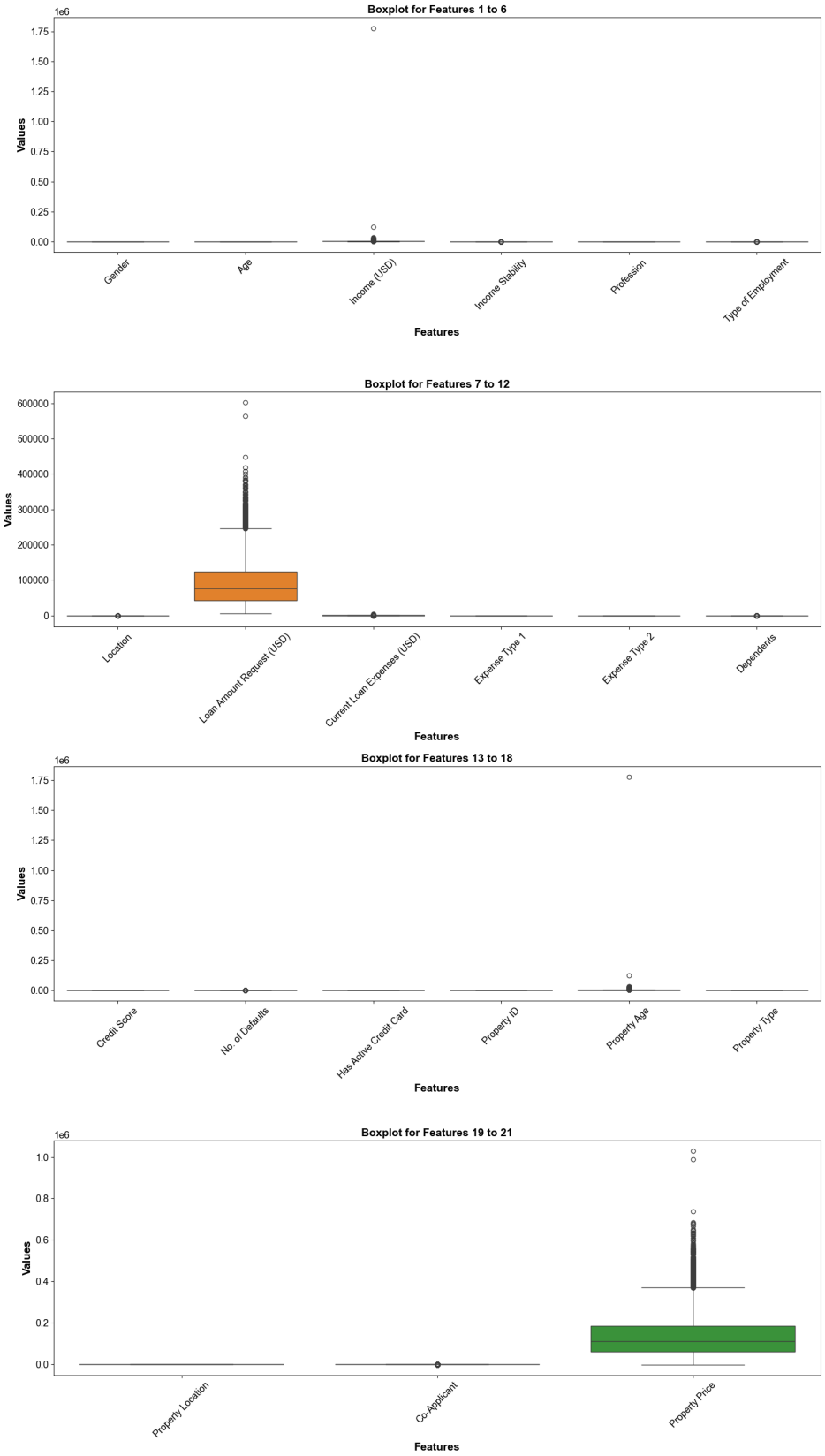

Figure 3: Box Plot for all features

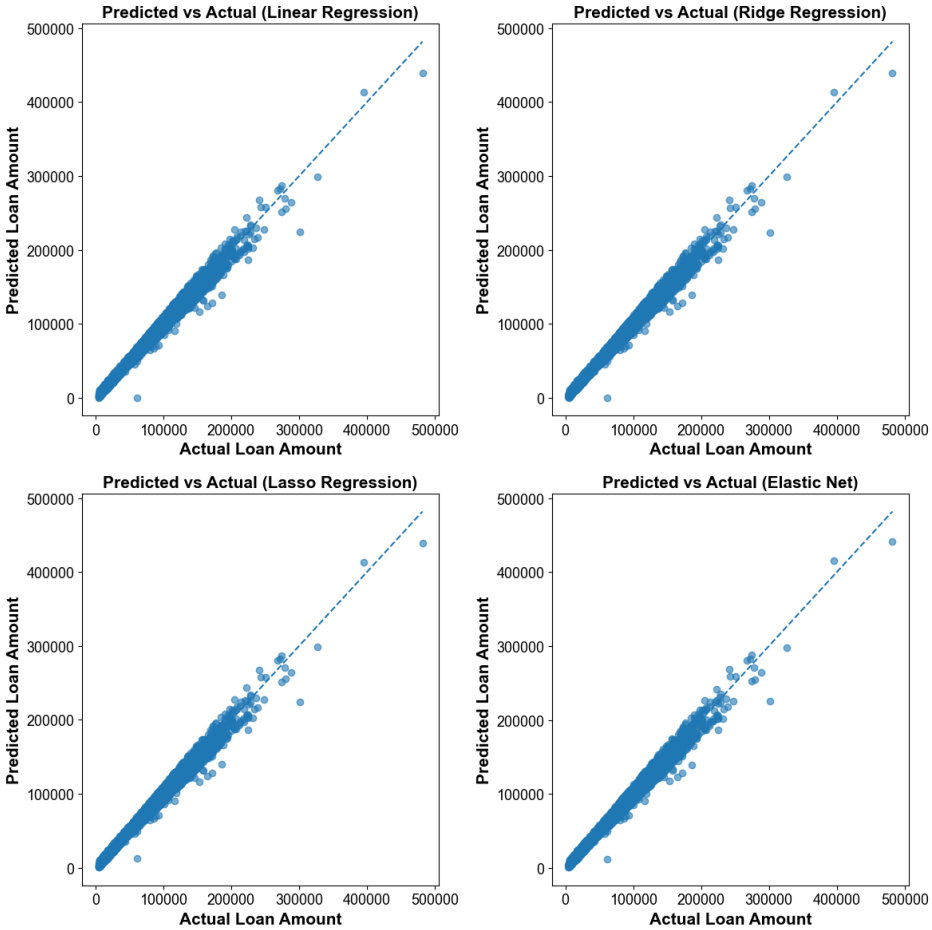

Figure 4: Predicted vs. actual values plot

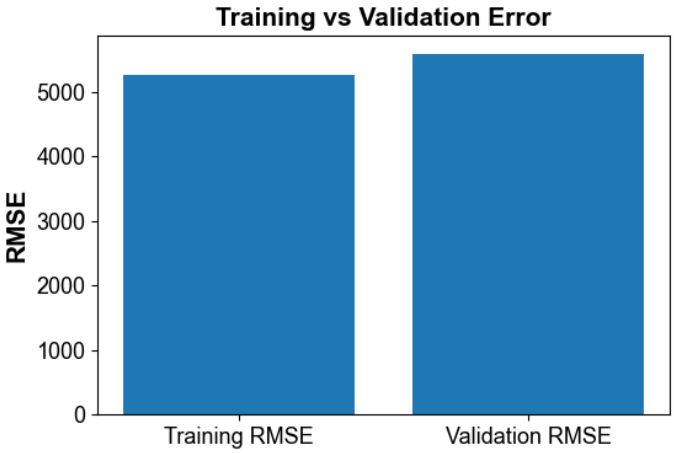

Figure 5: Training error vs. validation error plot

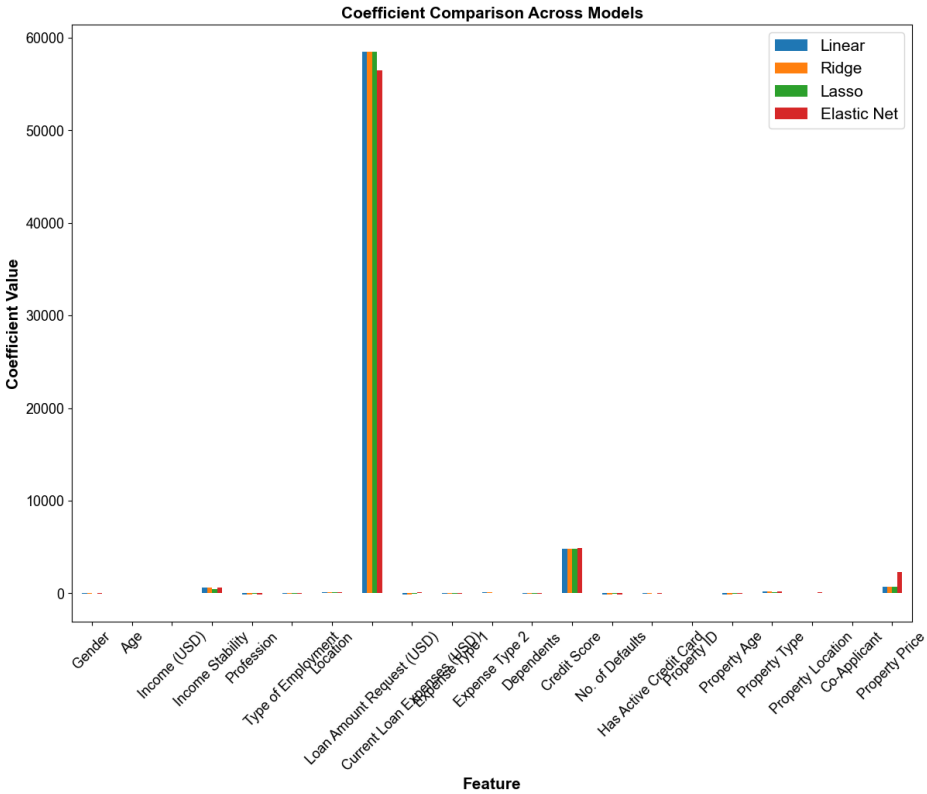

Figure 6: Coefficient comparison bar plot

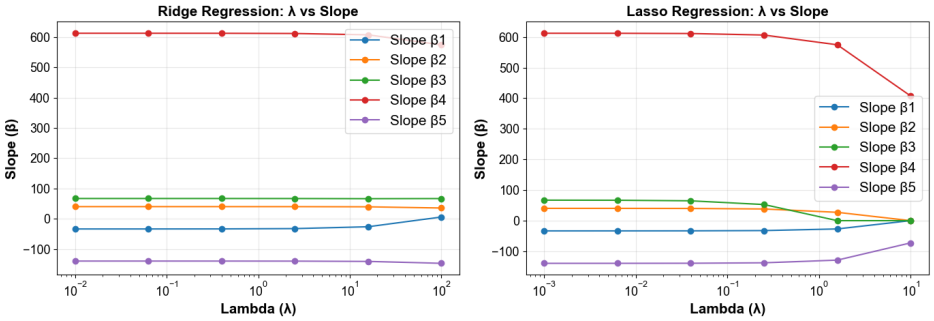

Figure 7: Lambda vs slope plot for Ridge and Lasso Regression

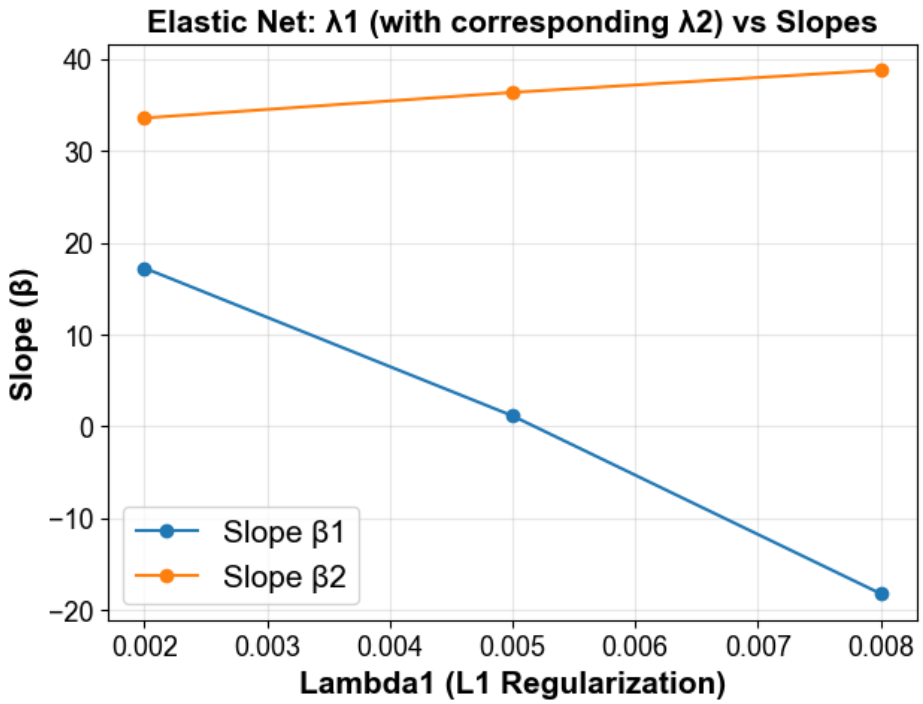

Figure 8: Lambda_1, Lambda_2, slope_1 and slope_2 plot for Elastic Regression

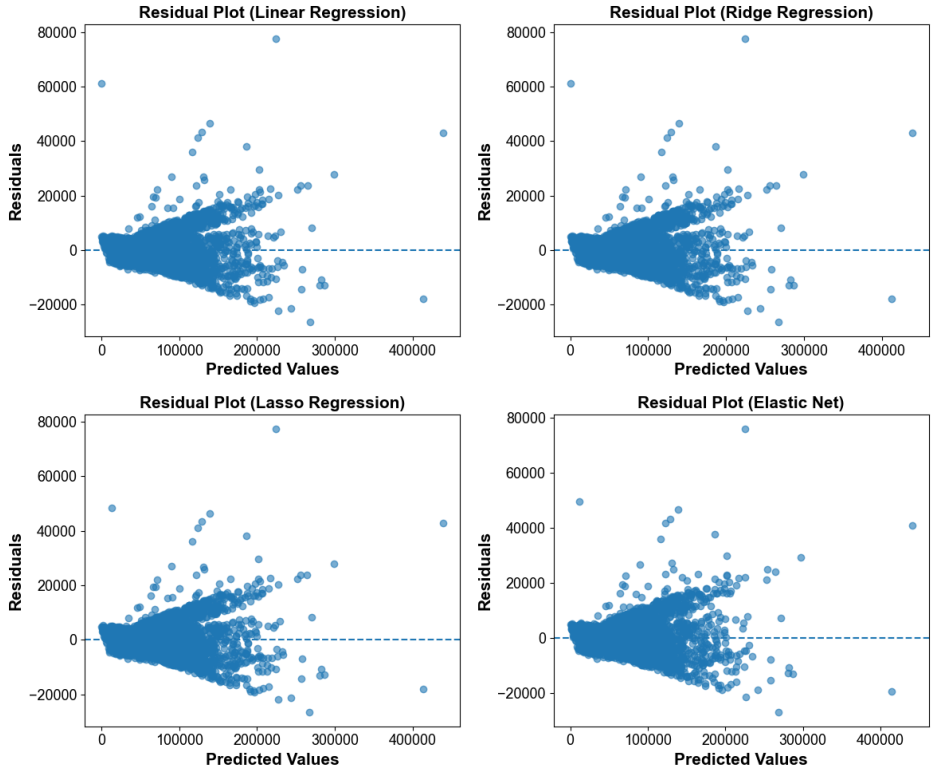

Figure 9: Residual plot

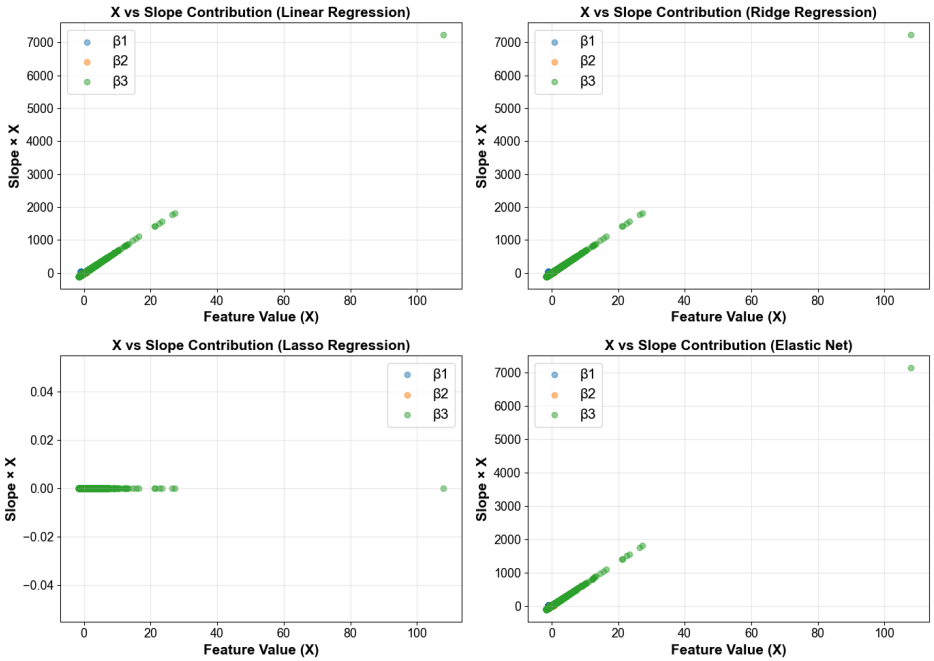

Figure 10: X vs Slope Plot for all regression.

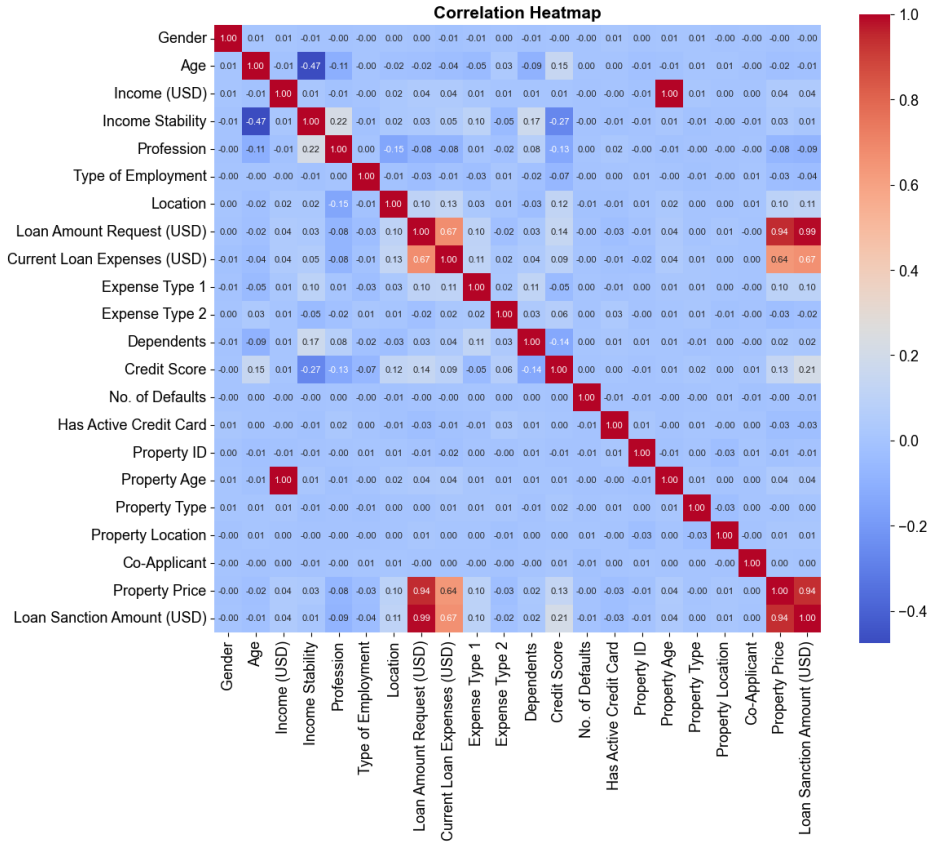

Figure 11: HeatMap for Correlation Analysis

6 Performance Tables

Hyperparameter Tuning Results

Table 1: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV R^2
Ridge Regression	Grid	'alpha': 0.1	0.9861
Lasso Regression	Grid	'alpha': 0.1	0.9861
Elastic Net Regression	Grid	'alpha': 0.1, 'L1 ratio': 0.8	0.9860

Cross-Validation Performance (K = 5)

Table 2: Cross-Validation Performance

Model	MAE	MSE	RMSE	R^2
Linear Regression	3744.43	3.12e+07	5584.21	0.9851
Ridge Regression	3744.42	3.12e+07	5584.15	0.9851
Lasso Regression	3737.25	3.08e+07	5553.88	0.9853
Elastic Net Regression	3755.10	3.12e+07	5585.98	0.9851

Test Set Performance Comparison

Table 3: Test Set Performance

Model	MAE	MSE	RMSE	R^2
Linear Regression	3744.43	3.12e+07	5584.21	0.9851
Ridge Regression	3744.42	3.12e+07	5584.15	0.9851
Lasso Regression	3737.25	3.08e+07	5553.88	0.9853
Elastic Net Regression	3755.10	3.12e+07	5585.98	0.9851

7 Effect of Regularization on Coefficients

Table 4: Coefficient Comparison

Feature	Linear	Ridge	Lasso	Elastic Net
Loan Amount Requested	58500	58300	58400	56500
Credit Score	4800	4700	4750	4600
Property Price	800	700	650	2200

8 Overfitting and Underfitting Analysis

- **Difference between training and validation errors:** Linear Regression shows a noticeable gap between training and validation performance, indicating sensitivity to noise and potential overfitting. Regularized models such as Ridge, Lasso, and Elastic Net reduce this gap by constraining model coefficients, leading to more stable validation performance.
- **Effect of regularization strength:** Increasing regularization strength (higher α) penalizes large coefficients, reducing model complexity. Ridge and Elastic Net demonstrate smoother predictions with controlled coefficients, while excessive regularization may lead to underfitting. Lasso may eliminate less important features entirely when regularization is strong.
- **Improvement in generalization after tuning:** Hyperparameter tuning using GridSearchCV and RandomizedSearchCV significantly improved generalization performance. Ridge and Elastic Net achieved higher validation R^2 scores compared to Linear Regression, indicating better balance between bias and variance.

9 Bias–Variance Analysis

- **Bias behavior of Linear Regression:** Linear Regression exhibits low bias but high variance, as it attempts to fit all features without any constraint. This makes it sensitive to fluctuations in training data, leading to unstable predictions on unseen data.
- **Variance reduction using Ridge and Elastic Net:** Ridge Regression and Elastic Net reduce variance by shrinking coefficient magnitudes. Elastic Net further balances Ridge and Lasso penalties, providing improved stability while retaining important features, resulting in better overall model robustness.
- **Feature sparsity effect in Lasso:** Lasso Regression introduces sparsity by forcing some coefficients to zero. This performs implicit feature selection, simplifying the model and reducing overfitting, but may slightly increase bias if important features are removed.

10 Observations & Conclusion

In this experiment, Linear Regression, Ridge Regression, Lasso Regression, and Elastic Net were implemented and evaluated. Linear Regression showed higher variance and weaker generalization due to the absence of regularization. Ridge Regression improved stability by penalizing large

coefficients, while Lasso enabled feature selection through coefficient sparsity. Elastic Net provided the best trade-off by combining Ridge and Lasso regularization, achieving superior generalization performance.

Optimal hyperparameters obtained through cross-validation balanced prediction accuracy and model complexity. Overall, Elastic Net emerged as the most reliable model, effectively minimizing overfitting while maintaining predictive accuracy.